

CS455: Introduction to Software Engineering

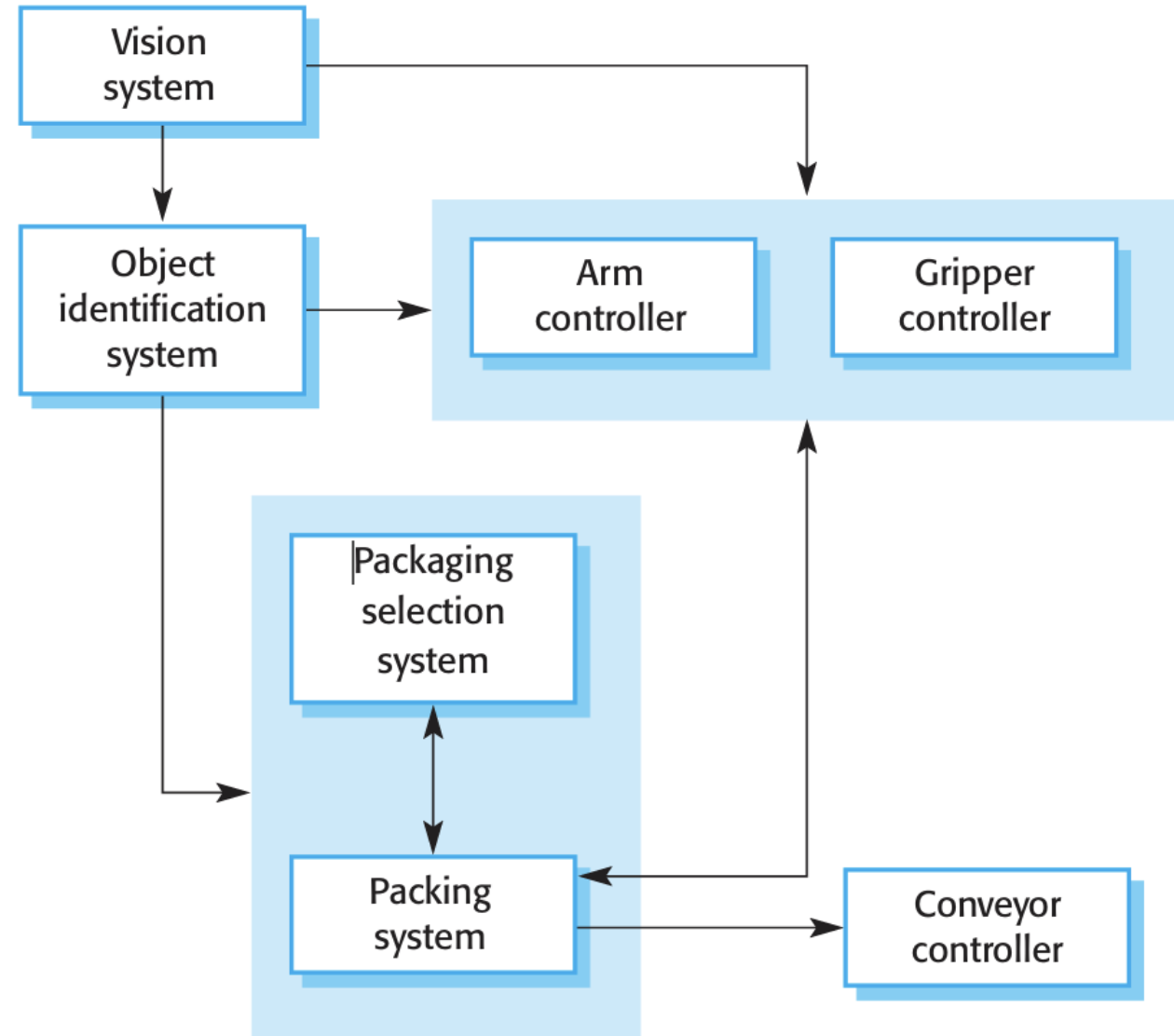
Lecture 8

Priyanka Bagade

Architectural Design

Architectural Design

- First stage in the software design process
- Identifies main structural components in the system and their relationships
- Critical link between specification and design phase
- Advantages of explicitly designing and documenting software architecture
 - Stakeholder Communication
 - System analysis
 - Large-scale reuse



Packing Robot Control System

Levels of Abstraction in Architectural Design

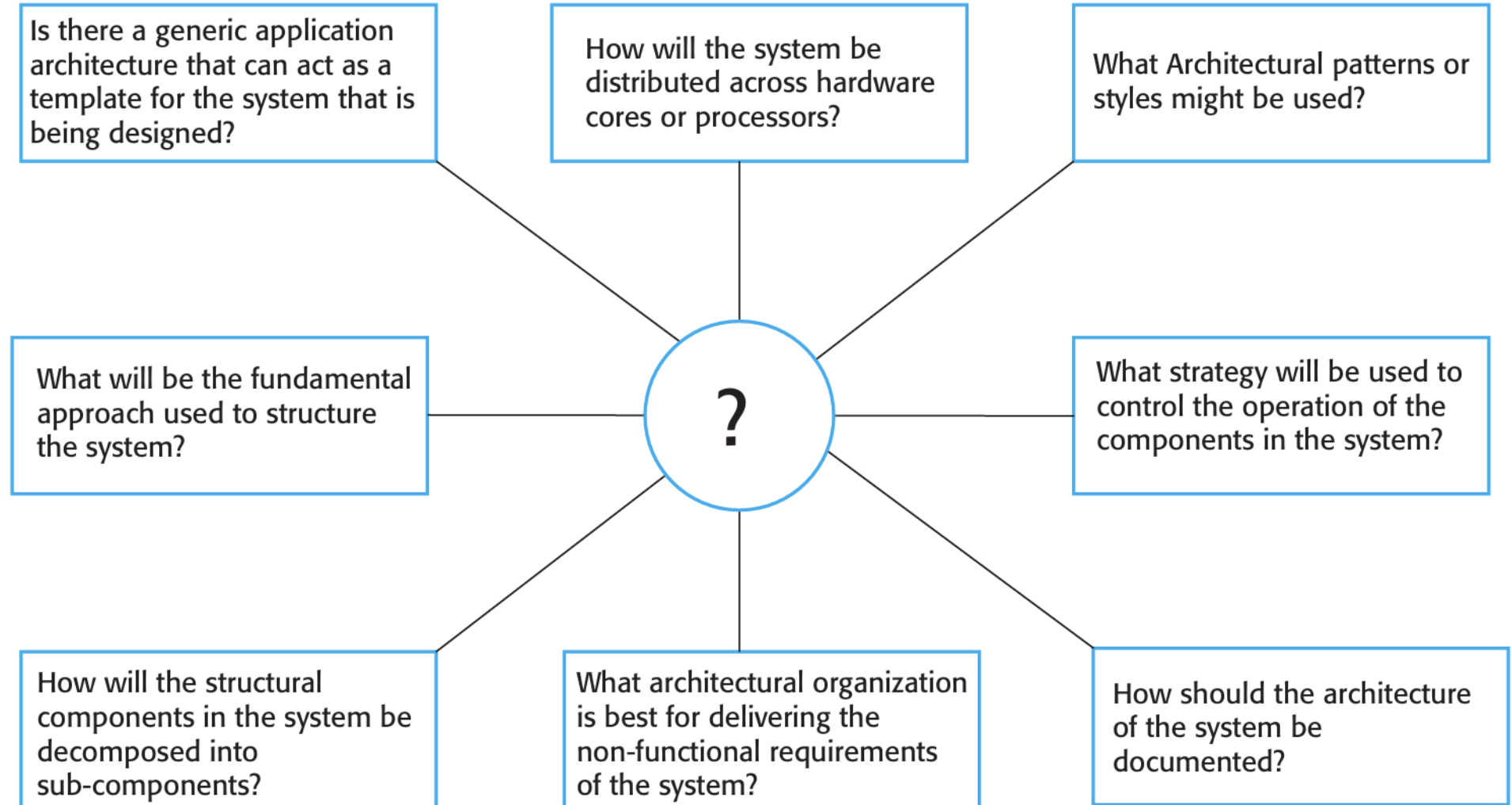
- Architecture in small
 - Includes architecture of individual programs.
 - At this level, we are concerned with the way that an individual program is decomposed into components.
- Architecture in large
 - includes architecture of complex enterprise systems that include other systems, programs, and program components.
 - These enterprise systems may be distributed over different computers, which may be owned and managed by different companies.

Usage of Architectural Model of the Program

- As a way of encouraging discussions about the system design
 - A high-level architectural view of a system is useful for communication with system stakeholders and project planning
 - Abstract view of the system
- As a way of documenting an architecture that has been designed
 - produce a complete system model that shows the different components in a system, their interfaces and their connections
 - a detailed architectural description makes it easier to understand and evolve the system

Architectural Design Decisions

- a creative process in which you design a system organization that will satisfy the functional and non-functional requirements of a system.

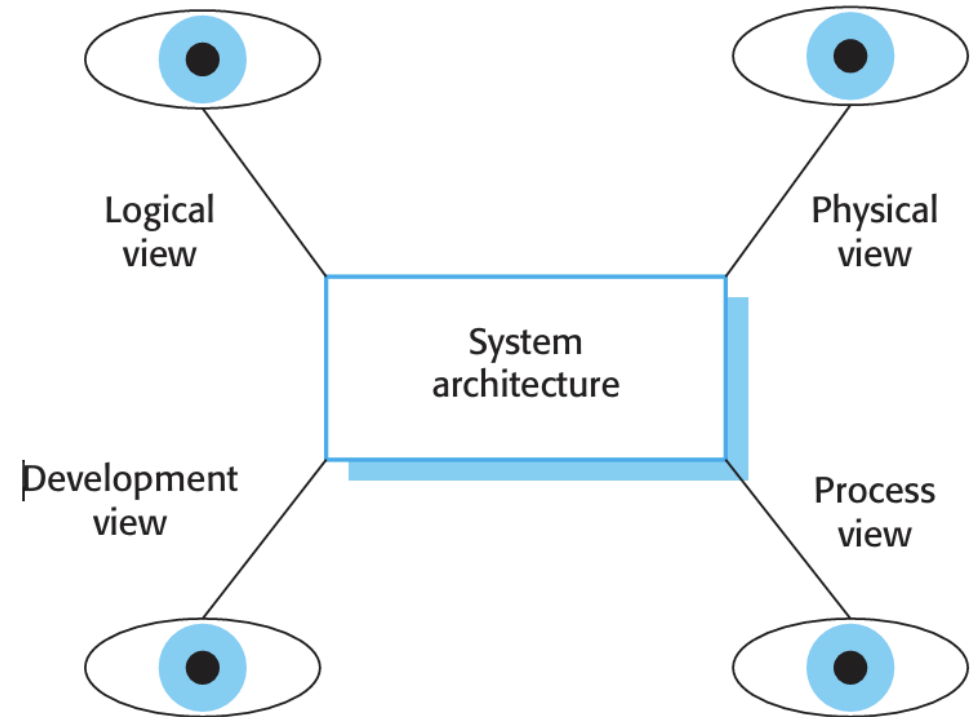


Architectural Design Decisions

- based on Non-functional requirements
 - **Performance** - localize critical operations within a small number of components, with these components deployed on the same computer rather than distributed across the network
 - **Security** - a layered structure for the architecture should be used
 - **Safety** - safety-related operations are co-located in a single component or in a small number of components
 - **Availability** - designed to include redundant components so that it is possible to replace and update components without stopping the system.
 - **Maintenance** - designed using fine-grain, self-contained components that may readily be changed.

Architectural Views

- Challenging to represent all relevant information about a system's architecture in a single diagram
- Krutchen (Krutchen 1995) suggested 4 +1 view model of software architecture.
- However, users of agile methods claim that detailed design documentation is mostly unused.



Architectural Patterns

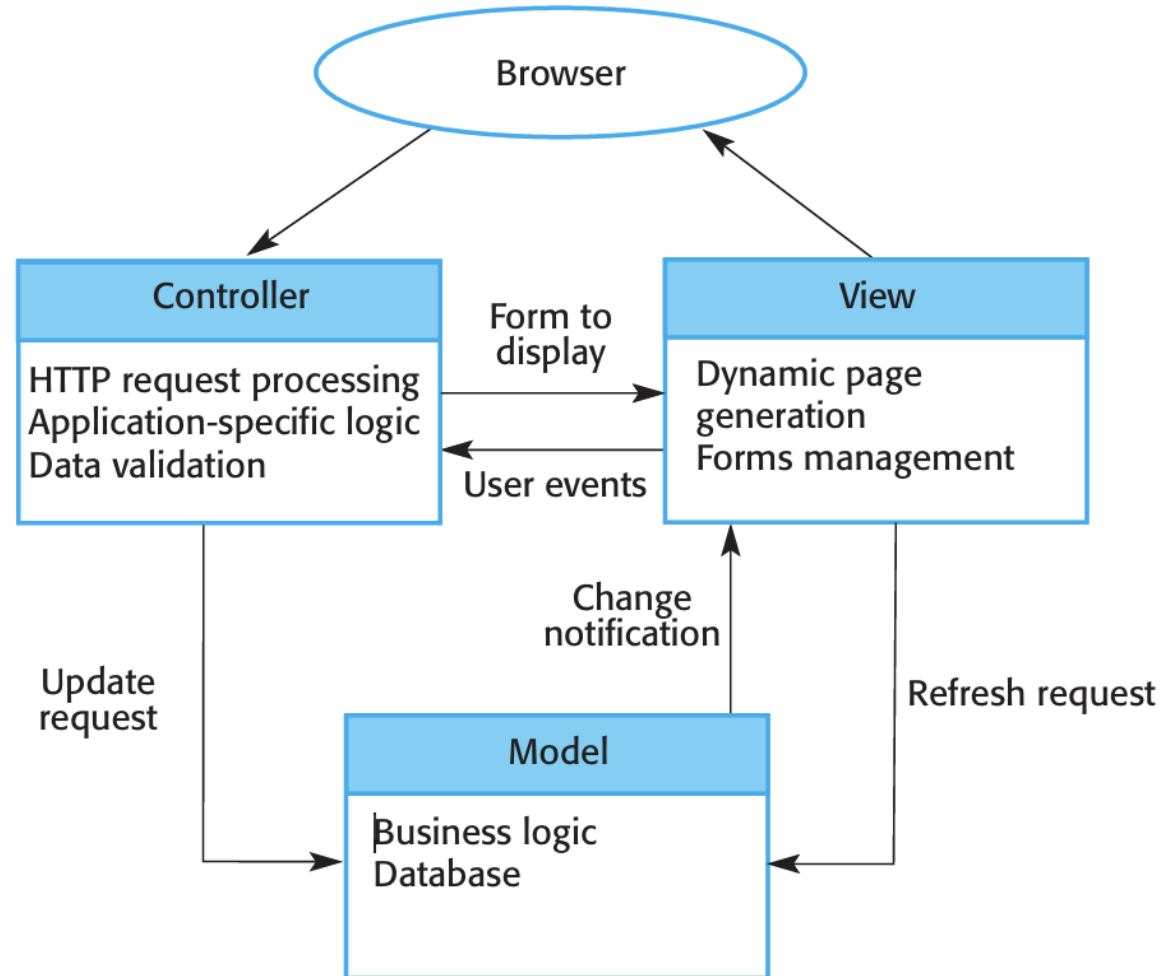
- The idea of patterns as a way of presenting, sharing, and reusing knowledge about software systems has been adopted in a number of areas of software engineering.
- Architectural patterns are a stylized, abstract description of good practice, which has been tried and tested in different systems and environments.
- Should describe strengths and weakness of the pattern

Architectural Patterns – MVC Pattern

- Model View Controller (MVC) pattern
 - the basis of interaction management in many web-based systems and is supported by most language frameworks
 - Includes the pattern name, a brief description, a graphical model, and an example of the type of system where the pattern is used
 - separates elements of a system, allowing them to change independently.

Name	MVC (Model-View-Controller)
Description	Separates presentation and interaction from the system data. The system is structured into three logical components that interact with each other. The Model component manages the system data and associated operations on that data. The View component defines and manages how the data is presented to the user. The Controller component manages user interaction (e.g., key presses, mouse clicks, etc.) and passes these interactions to the View and the Model. See Figure 6.5.
Example	Figure 6.6 shows the architecture of a web-based application system organized using the MVC pattern.
When used	Used when there are multiple ways to view and interact with data. Also used when the future requirements for interaction and presentation of data are unknown.
Advantages	Allows the data to change independently of its representation and vice versa. Supports presentation of the same data in different ways, with changes made in one representation shown in all of them.
Disadvantages	May involve additional code and code complexity when the data model and interactions are simple.

Web application architecture using the MVC pattern



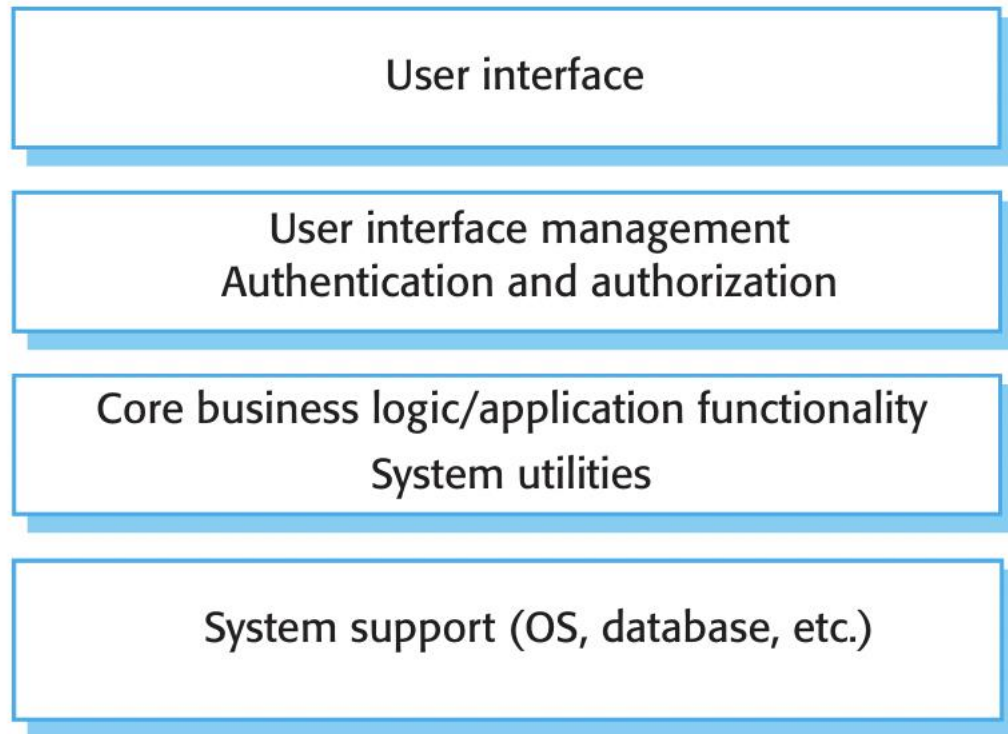
Architectural Patterns – Layered Pattern

- another way of achieving separation and independence, similar to MVC
- conceptual organization of a system
- system functionality is organized into separate layers, and each layer only relies on the facilities and services offered by the layer immediately beneath it
- supports the incremental development of systems
- If its interface is unchanged, a new layer with extended functionality can replace an existing layer without changing other parts of the system.

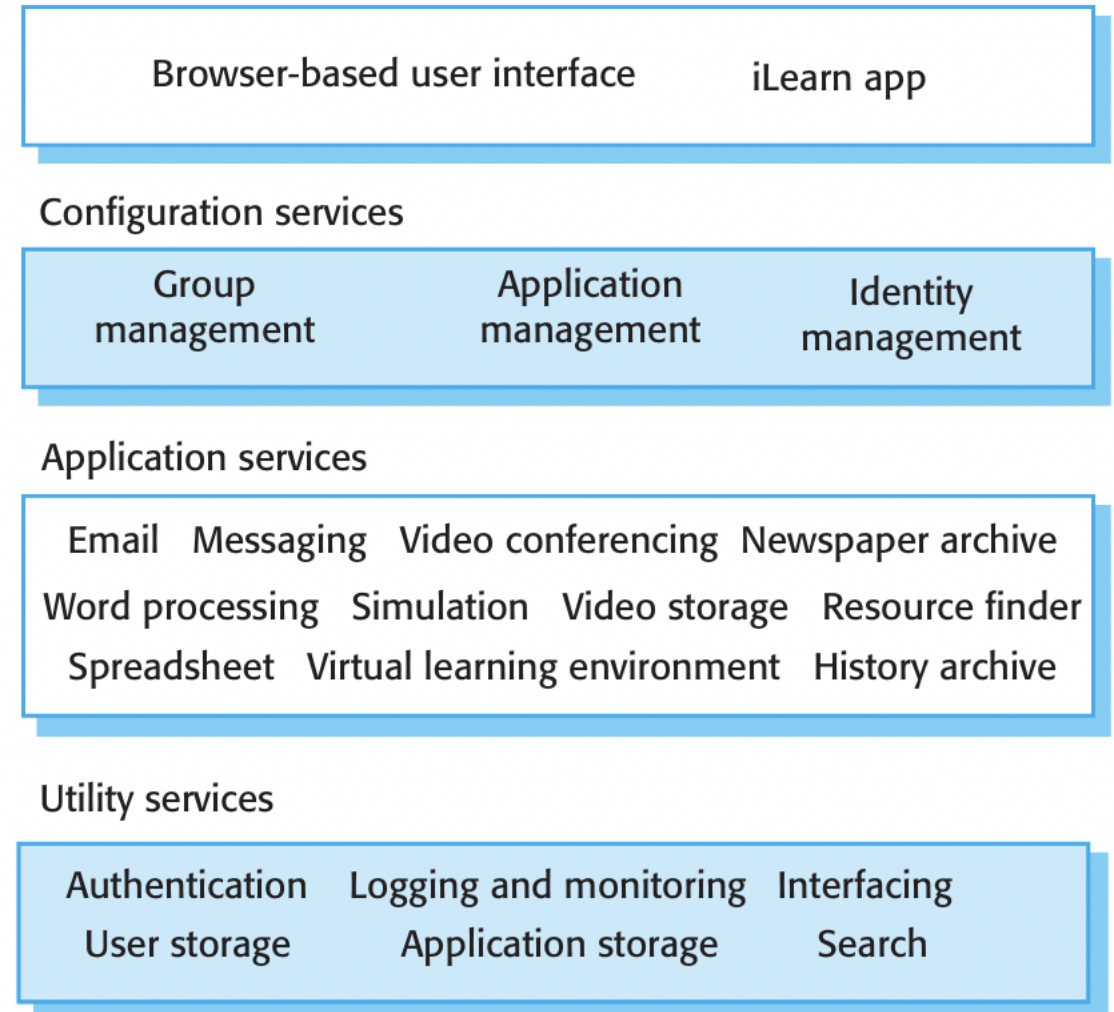
Layered Pattern

Name	Layered architecture
Description	Organizes the system into layers, with related functionality associated with each layer. A layer provides services to the layer above it, so the lowest level layers represent core services that are likely to be used throughout the system. See Figure 6.8.
Example	A layered model of a digital learning system to support learning of all subjects in schools (Figure 6.9).
When used	Used when building new facilities on top of existing systems; when the development is spread across several teams with each team responsibility for a layer of functionality; when there is a requirement for multilevel security.
Advantages	Allows replacement of entire layers as long as the interface is maintained. Redundant facilities (e.g., authentication) can be provided in each layer to increase the dependability of the system.
Disadvantages	In practice, providing a clean separation between layers is often difficult, and a high-level layer may have to interact directly with lower-level layers rather than through the layer immediately below it. Performance can be a problem because of multiple levels of interpretation of a service request as it is processed at each layer.

Layered Pattern - Example



Generic Example



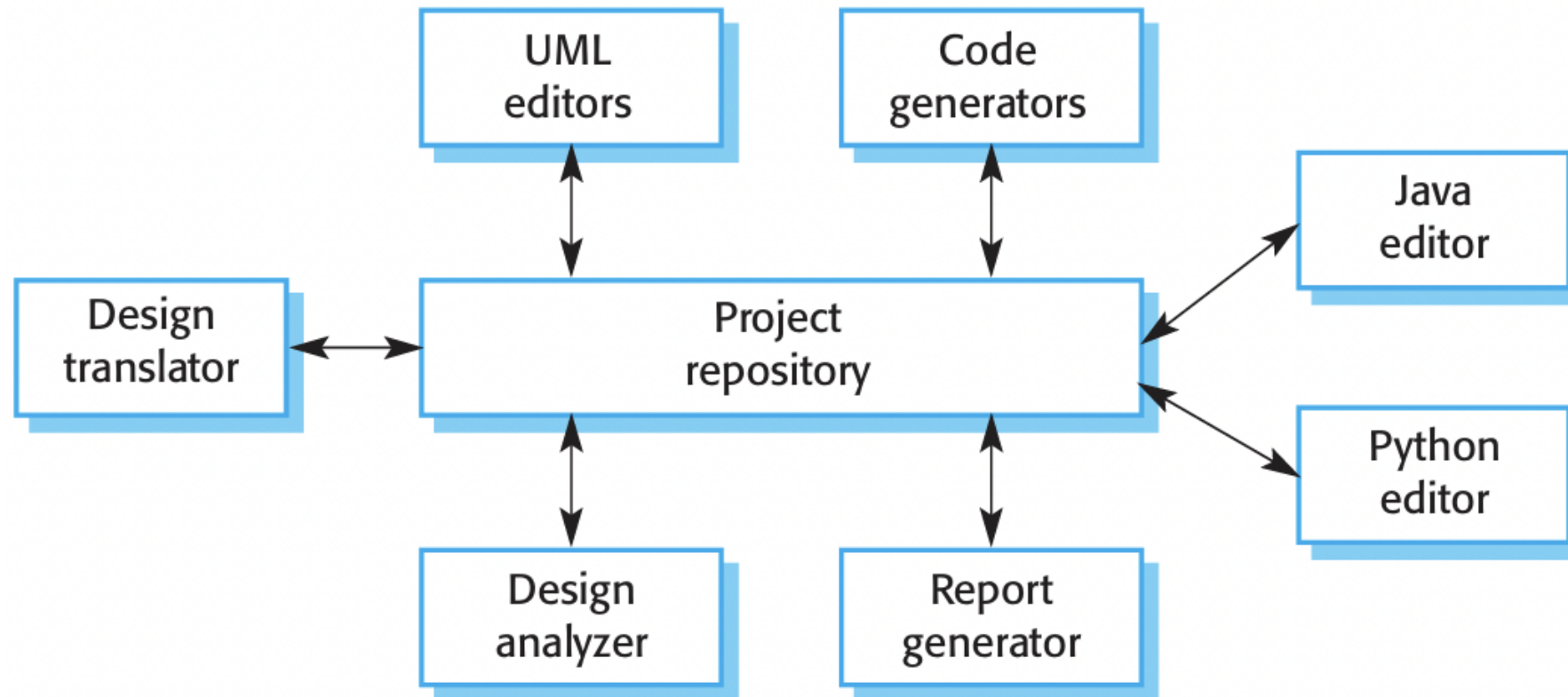
iLearn System Architecture

Architectural Patterns – Repository Pattern

- describes how a set of interacting components can share data
- suited to applications in which data generated by one component and used by another
- the repository is passive and control is the responsibility of the components using the repository
- concerned with the static structure of a system and does not show its runtime organization

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.11 is an example of an IDE where the components use a repository of system design information. Each software tool generates information, which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent; they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

A repository architecture for an IDE



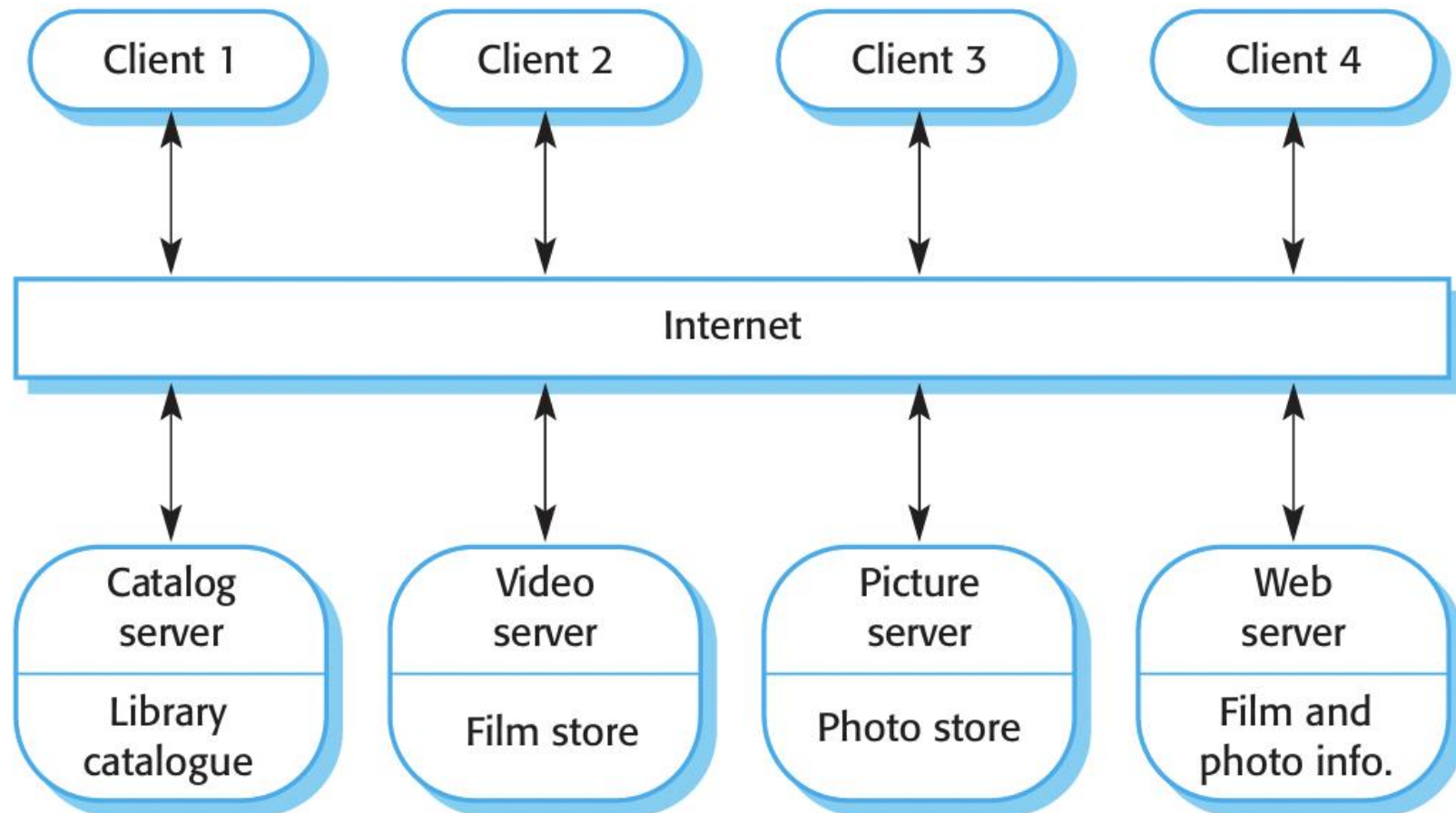
Architectural Patterns – Client Server Pattern

- A system that follows the Client–Server pattern is organized as a set of services and associated servers, and clients that access and use the services.
 - A set of servers that offer services to other components
 - A set of clients that call on the services offered by servers.
 - A network that allows the clients to access these services.
- usually thought of as distributed systems architectures, but the logical model of independent services running on separate servers can be implemented on a single computer

Client Server Pattern

Name	Client-server
Description	In a client-server architecture, the system is presented as a set of services, with each service delivered by a separate server. Clients are users of these services and access servers to make use of them.
Example	Figure 6.13 is an example of a film and video/DVD library organized as a client-server system.
When used	Used when data in a shared database has to be accessed from a range of locations. Because servers can be replicated, may also be used when the load on a system is variable.
Advantages	The principal advantage of this model is that servers can be distributed across a network. General functionality (e.g., a printing service) can be available to all clients and does not need to be implemented by all services.
Disadvantages	Each service is a single point of failure and so is susceptible to denial-of-service attacks or server failure. Performance may be unpredictable because it depends on the network as well as the system. Management problems may arise if servers are owned by different organizations.

Client Server Pattern – Example of film Library



Architectural Patterns – Pipe filter Pattern

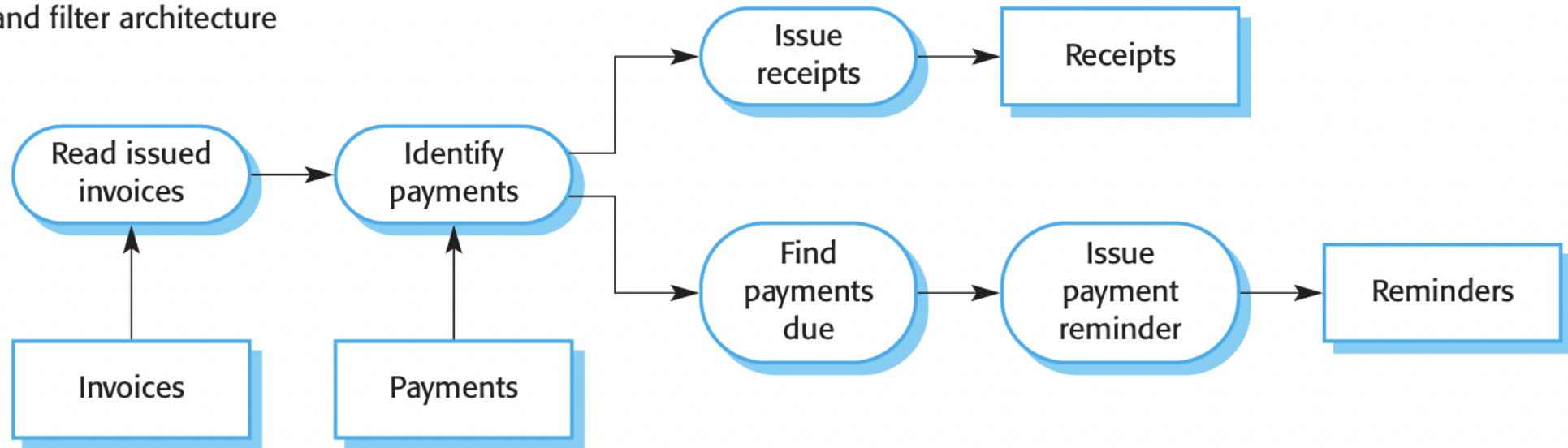
- a model of the runtime organization of a system where functional transformations process their inputs and produce outputs.
- The name “pipe and filter” comes from the original Unix system where it was possible to link processes using “pipes.”
- Data flows from one to another and is transformed as it moves through the sequence.
- Each processing step is implemented as a transform.

Pipe - filter Pattern

Name	Pipe and filter
Description	The processing of the data in a system is organized so that each processing component (filter) is discrete and carries out one type of data transformation. The data flows (as in a pipe) from one component to another for processing.
Example	Figure 6.15 is an example of a pipe and filter system used for processing invoices.
When used	Commonly used in data-processing applications (both batch and transaction-based) where inputs are processed in separate stages to generate related outputs.
Advantages	Easy to understand and supports transformation reuse. Workflow style matches the structure of many business processes. Evolution by adding transformations is straightforward. Can be implemented as either a sequential or concurrent system.
Disadvantages	The format for data transfer has to be agreed between communicating transformations. Each transformation must parse its input and unparse its output to the agreed form. This increases system overhead and may mean that it is impossible to reuse architectural components that use incompatible data structures.

Example

Figure 6.15 An example of the pipe and filter architecture



References

- Chapter 6, Architectural Design: [Software Engineering 10e Edition by Ian Sommerville](#)